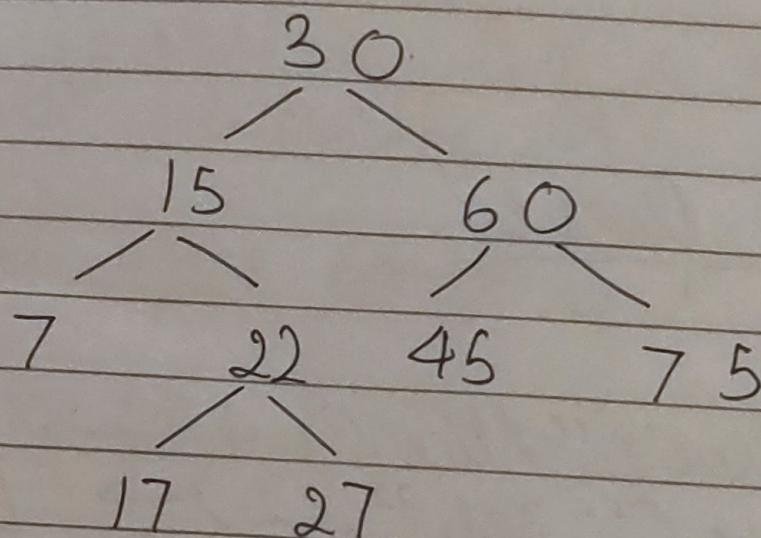


1.0 BST Traversal

1.0.1 Drawing of BST

Insert Order as given



This seems balanced

$$BF = \text{height}(L) - \text{height}(R)$$

1

1.0.2 in-order - 7, 15, 17, 22, 27, 30, 45, 60, 75

Pre-order - 30, 15, 7, 22, 17, 27, 60, 45, 75

Post-order - 7, 17, 27, 22, 15, 45, 75, 60, 30

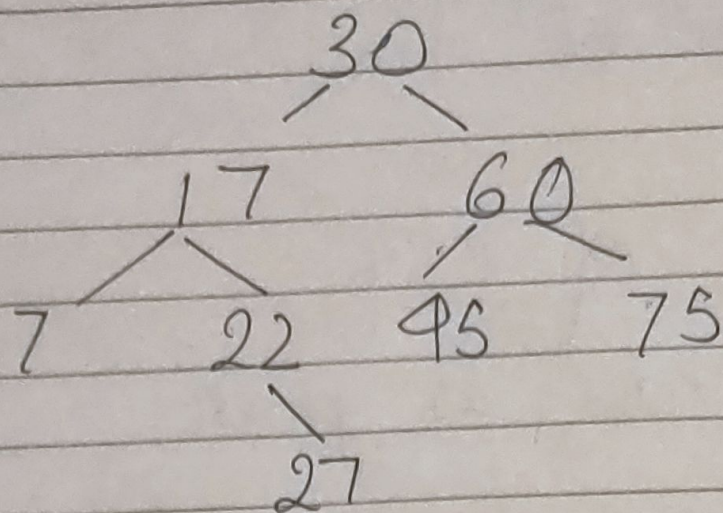
1.0.3 15 has 2 children nodes.

We need to find its order successor.

So we need to replace 15 with 17 and delete 17

Also link 27 back to it

We need to do this cause direct deletion would break the BST structure.



20 Priority Queue

- 201
1. enqueue("Alice", 4) → [Alice, 4]
 2. enqueue("Bob", 2) → [(Bob, 2), (Alice, 4)]
 3. enqueue("Carol", 5) → [(Bob, 2), (Alice, 4), (Carol, 5)]
 4. enqueue("Dave", 1) → [(Dave, 1), (Bob, 2), (Alice, 4), (Carol, 5)]
 5. dequeue() → [(Bob, 2), (Alice, 4), (Carol, 5)]
 6. enqueue("Eve", 3) → [(Bob, 2), (Eve, 3), (Alice, 4), (Carol, 5)]
 7. dequeue() → [(Eve, 3), (Alice, 4), (Carol, 5)]

The current Priority Order final is

Eve → Alice → Carol

202 A binary ~~tree~~ heap maintains a complete binary stored in an array.

- Insertion → Add at the end, then up-heap
swap with parent until has to $O(\log n) \therefore O(\log n) \leq O(\log n)$
- Delete top - Replace top with the last and then shrink. this also takes $O(\log n)$ swaps.

//_

3^o How does this transformation affect the original BST Properties.

Due to transformation the value increases during traversal, so nodes become greater, some nodes might become greater.

∴ The structure is still a ~~BST~~, Binary Tree but not necessarily BST

Time And Space Complexity

Time is just $O(n)$ as we visit every node once

Space should be $O(h)$ where h is height of tree, in normal cases $h \approx \log n$ but for the worst skewed tree it could be that $h = n$

Some optimisation can be replacement of recursion with stacks, but it would be a manual addition via linked list.

Rebalancing of the final prosperity tree ~~cause~~ for future search needs.

One that I found online was Morris traversal, but would be really hard to implement.

4. Yes there is a better approach known as degree based approach.
this method should be able to parse in $O(n)$ time, by maintaining a double ended queue whose indices are in decreasing order.

At each step outdated indices are popped from the front and smaller values from the back before adding new index.

Each element is added and removed at most once, giving linear overall work

Pseudocode.

```
def SkyMan(arr, k)
    initialize dq, list, result
    for i in arr
        while dq != NULL, dq.front() ≤ i - k
            dq.pop-front()

        while dq != and arr[i] ≥ arr[dq.back()]
            dq.pop-back()

        dq.push-back(i)

        if i ≥ k - 1
            result.append(arr[dq.front()])

    return result.
```